

Section 77 Editorial — Suffix Automata And Palindromic Trees

Each solution starts from the uncompressed object, identifies the bottleneck, and states the invariant that justifies the compressed structure.

A. K-th Distinct Substring

How to find it

View distinct substrings as nonempty paths from the SAM root. Count paths in reverse length order, then skip lexicographic transition blocks.

Correctness

Each distinct substring spells exactly one root path. Each outgoing transition partitions paths by their first next character, so block subtraction selects exactly the k-th path.

Complexity

$O(n \log \text{alphabet})$ time and $O(n \cdot \text{alphabet})$ memory in the fixed-array implementation.

Reconstruction trace and traps

An outgoing letter block contains the one substring ending after that edge and all extensions below it, hence size $1 + \text{dp}[\text{next}]$. Iterate letters in order, subtract entire blocks, append the selected letter, and stop when the remaining rank is one. Otherwise decrement once and continue below the edge.

Do not multiply by occurrence count in this distinct-substring problem. Cap DP counts before overflow and report impossibility when k exceeds $\text{dp}[\text{root}]$.

Reference C++

```
#include <bits/stdc++.h>
using namespace std;
struct SAM {
    struct Node {
        array<int, 26> next;
        int link = -1, len = 0;
        long long occ = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> st;
    int last = 0;
    SAM() { st.emplace_back(); }
    void add(int c) {
        int cur = st.size();
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].occ = 1;
        int p = last;
        while (p != -1 && st[p].next[c] == -1)
```

```

    st[p].next[c] = cur, p = st[p].link;
if (p == -1)
    st[cur].link = 0;
else {
    int q = st[p].next[c];
    if (st[p].len + 1 == st[q].len)
        st[cur].link = q;
    else {
        int clone = st.size();
        st.push_back(st[q]);
        st[clone].len = st[p].len + 1;
        st[clone].occ = 0;
        while (p != -1 && st[p].next[c] == q)
            st[p].next[c] = clone, p = st[p].link;
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}
SAM(const string &s) : SAM() {
    for (char c : s)
        add(c - 'a');
}
vector<int> order() const {
    vector<int> o(st.size());
    iota(o.begin(), o.end(), 0);
    sort(o.begin(), o.end(),
        [&](int a, int b) { return st[a].len < st[b].len; });
    return o;
}
void occurrences() {
    auto o = order();
    for (int i = (int)o.size() - 1; i; --i)
        st[st[o[i]].link].occ += st[o[i]].occ;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    long long k;
    cin >> s >> k;
    SAM a(s);
    const long long CAP = 4000000000000000LL;
    auto o = a.order();
    vector<long long> dp(a.st.size());
    for (int z = (int)o.size() - 1; z >= 0; --z) {
        int v = o[z];
        for (int u : a.st[v].next)
            if (u != -1) {
                long long add = min(CAP, 1 + dp[u]);
                dp[v] = dp[v] >= CAP - add ? CAP : dp[v] + add;
            }
    }
    if (k > dp[0]) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    string ans;
    int v = 0;
    while (k) {
        for (int c = 0; c < 26; ++c) {

```

```

int u = a.st[v].next[c];
if (u == -1)
    continue;
long long block = 1 + dp[u];
if (k > block)
    k -= block;
else {
    ans += char('a' + c);
    --k;
    v = u;
    break;
}
}
}
cout << ans << '\n';
}

```

Reference Python

```

class SAM:
    def __init__(self, s):
        self.next = [{}]
        self.link = [-1]
        self.length = [0]
        self.occ = [0]
        last = 0
        for char in s:
            current = len(self.next)
            self.next.append({})
            self.link.append(0)
            self.length.append(self.length[last] + 1)
            self.occ.append(1)
            parent = last
            while parent != -1 and char not in self.next[parent]:
                self.next[parent][char] = current
                parent = self.link[parent]
            if parent == -1:
                self.link[current] = 0
            else:
                target = self.next[parent][char]
                if self.length[parent] + 1 == self.length[target]:
                    self.link[current] = target
                else:
                    clone = len(self.next)
                    self.next.append(self.next[target].copy())
                    self.link.append(self.link[target])
                    self.length.append(self.length[parent] + 1)
                    self.occ.append(0)
                    while (
                        parent != -1 and self.next[parent].get(char) == target
                    ):
                        self.next[parent][char] = clone
                        parent = self.link[parent]
                    self.link[target] = self.link[current] = clone
            last = current

    def order(self):
        return sorted(range(len(self.next)), key=self.length.__getitem__)

    def occurrences(self):
        order = self.order()

```

```

        for state in reversed(order[1:]):
            self.occ[self.link[state]] += self.occ[state]

def main():
    import sys

    raw_s, raw_k = sys.stdin.buffer.read().split()
    automaton = SAM(raw_s.decode())
    k = int(raw_k)
    paths = [0] * len(automaton.next)
    for state in reversed(automaton.order()):
        paths[state] = sum(
            1 + paths[target] for target in automaton.next[state].values()
        )
    if k > paths[0]:
        print("IMPOSSIBLE")
        return
    answer = []
    state = 0
    while k:
        for char, target in sorted(automaton.next[state].items()):
            block = 1 + paths[target]
            if k > block:
                k -= block
            else:
                answer.append(char)
                k -= 1
                state = target
                break
    print("".join(answer))

if __name__ == "__main__":
    main()

```

B. Longest Common Substring

How to find it

Build a SAM for the first string and maintain the longest suffix of each scanned prefix of the second string that the automaton accepts.

Correctness

The scan invariant gives the longest common substring ending at every position of the second string. Taking the maximum therefore considers every possible endpoint.

Complexity

$O(|a|+|b|)$ time and $O(|a| \cdot \text{alphabet})$ memory.

State repair and tests

On a missing edge, follow suffix links until an edge exists or the root is reached. After a retreat, set the current matched length to at most the new state's `len`; then consume the character. The maximum over all scanned positions considers every possible endpoint in the second string.

Test no common character, complete containment, repeated borders, and a match that survives several suffix-link retreats. Resetting length to zero after every missing transition loses valid shorter suffix matches.

Reference C++

```
#include <bits/stdc++.h>
using namespace std;
struct SAM {
    struct Node {
        array<int, 26> next;
        int link = -1, len = 0;
        long long occ = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> st;
    int last = 0;
    SAM() { st.emplace_back(); }
    void add(int c) {
        int cur = st.size();
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].occ = 1;
        int p = last;
        while (p != -1 && st[p].next[c] == -1)
            st[p].next[c] = cur, p = st[p].link;
        if (p == -1)
            st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len)
                st[cur].link = q;
            else {
                int clone = st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;
                st[clone].occ = 0;
                while (p != -1 && st[p].next[c] == q)
                    st[p].next[c] = clone, p = st[p].link;
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
    SAM(const string &s) : SAM() {
        for (char c : s)
            add(c - 'a');
    }
    vector<int> order() const {
        vector<int> o(st.size());
        iota(o.begin(), o.end(), 0);
        sort(o.begin(), o.end(),
            [&](int a, int b) { return st[a].len < st[b].len; });
        return o;
    }
    void occurrences() {
        auto o = order();
        for (int i = (int)o.size() - 1; i; --i)
            st[st[o[i]].link].occ += st[o[i]].occ;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string a, b;
    cin >> a >> b;
```

```

SAM s(a);
int v = 0, len = 0, ans = 0;
for (char ch : b) {
    int c = ch - 'a';
    while (v && s.st[v].next[c] == -1)
        v = s.st[v].link, len = min(len, s.st[v].len);
    if (s.st[v].next[c] != -1)
        v = s.st[v].next[c], ++len;
    else
        v = 0, len = 0;
    ans = max(ans, len);
}
cout << ans << '\n';
}

```

Reference Python

```

class SAM:
    def __init__(self, s):
        self.next = [{}]
        self.link = [-1]
        self.length = [0]
        self.occ = [0]
        last = 0
        for char in s:
            current = len(self.next)
            self.next.append({})
            self.link.append(0)
            self.length.append(self.length[last] + 1)
            self.occ.append(1)
            parent = last
            while parent != -1 and char not in self.next[parent]:
                self.next[parent][char] = current
                parent = self.link[parent]
            if parent == -1:
                self.link[current] = 0
            else:
                target = self.next[parent][char]
                if self.length[parent] + 1 == self.length[target]:
                    self.link[current] = target
                else:
                    clone = len(self.next)
                    self.next.append(self.next[target].copy())
                    self.link.append(self.link[target])
                    self.length.append(self.length[parent] + 1)
                    self.occ.append(0)
                    while (
                        parent != -1 and self.next[parent].get(char) == target
                    ):
                        self.next[parent][char] = clone
                        parent = self.link[parent]
                    self.link[target] = self.link[current] = clone
            last = current

    def order(self):
        return sorted(range(len(self.next)), key=self.length.__getitem__)

    def occurrences(self):
        order = self.order()
        for state in reversed(order[1:]):
            self.occ[self.link[state]] += self.occ[state]

```

```

def main():
    import sys

    a, b = (part.decode() for part in sys.stdin.buffer.read().split())
    automaton = SAM(a)
    state = length = answer = 0
    for char in b:
        while state and char not in automaton.next[state]:
            state = automaton.link[state]
            length = min(length, automaton.length[state])
        if char in automaton.next[state]:
            state = automaton.next[state][char]
            length += 1
        else:
            state = length = 0
        answer = max(answer, length)
    print(answer)

if __name__ == "__main__":
    main()

```

C. Palindrome Prefix Profile

How to find it

A newly appended character can introduce only one new distinct palindrome: the longest palindromic suffix. Maintain it with palindromic suffix links.

Correctness

Every new occurrence ends at the appended position. Shorter palindromic suffixes already occurred as suffixes of the previous longest one; creation of exactly the missing longest node is therefore necessary and sufficient.

Complexity

$O(n \cdot \text{alphabet})$ stored transitions and amortized $O(n)$ traversal.

Creation invariant and tests

After appending at position i , follow links until the characters surrounding the candidate suffix agree. If its transition already exists, no distinct node is created. Otherwise create the new longest palindromic suffix and find its proper palindromic suffix link with the same search from the parent's link.

The -1 and 0 roots must never be counted as text palindromes. Test one character, even palindromes, nested odd palindromes, and repeated occurrences that must reuse an existing node.

Reference C++

```

#include <bits/stdc++.h>
using namespace std;
struct Eertree {
    struct N {
        array<int, 26> to;
        int len = 0, link = 0;
        long long occ = 0;
        N() { to.fill(0); }
    };

```

```

};
vector<N> t;
string s;
int last = 1;
Eertree() {
    t.resize(2);
    t[0].len = -1;
    t[0].link = 0;
    t[1].len = 0;
    t[1].link = 0;
}
bool add(char ch) {
    int c = ch - 'a', pos = s.size();
    s += ch;
    int p = last;
    while (pos - 1 - t[p].len < 0 || s[pos - 1 - t[p].len] != ch)
        p = t[p].link;
    if (t[p].to[c]) {
        last = t[p].to[c];
        ++t[last].occ;
        return false;
    }
    int v = t.size();
    t.emplace_back();
    t[v].len = t[p].len + 2;
    t[p].to[c] = v;
    if (t[v].len == 1)
        t[v].link = 1;
    else {
        int q = t[p].link;
        while (pos - 1 - t[q].len < 0 || s[pos - 1 - t[q].len] != ch)
            q = t[q].link;
        t[v].link = t[q].to[c];
    }
    last = v;
    t[v].occ = 1;
    return true;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    Eertree e;
    for (char c : s) {
        e.add(c);
        cout << e.t.size() - 2 << ' ' << e.t[e.last].len << '\n';
    }
}

```

Reference Python

```

class Eertree:
    def __init__(self):
        self.next = [{}, {}]
        self.length = [-1, 0]
        self.link = [0, 0]
        self.occ = [0, 0]
        self.text = []

```

```

self.last = 1

def add(self, char):
    position = len(self.text)
    self.text.append(char)
    node = self.last
    while (
        position - 1 - self.length[node] < 0
        or self.text[position - 1 - self.length[node]] != char
    ):
        node = self.link[node]
    if char in self.next[node]:
        self.last = self.next[node][char]
        self.occ[self.last] += 1
        return False
    created = len(self.next)
    self.next.append({})
    self.length.append(self.length[node] + 2)
    self.link.append(1)
    self.occ.append(1)
    self.next[node][char] = created
    if self.length[created] > 1:
        suffix = self.link[node]
        while (
            position - 1 - self.length[suffix] < 0
            or self.text[position - 1 - self.length[suffix]] != char
        ):
            suffix = self.link[suffix]
        self.link[created] = self.next[suffix][char]
    self.last = created
    return True

def main():
    import sys

    tree = Eertree()
    output = []
    for char in sys.stdin.buffer.readline().decode().strip():
        tree.add(char)
        output.append(f"{len(tree.next) - 2} {tree.length[tree.last]}")
    print("\n".join(output))

if __name__ == "__main__":
    main()

```

D. Shortest Absent Word

How to find it

A word is a substring exactly when it labels a SAM root path. Breadth-first search those paths in letter order and stop at the first missing transition.

Correctness

BFS processes path labels by increasing length and, within a level, lexicographic order. A missing next edge certifies absence, so the first such label meets both tie-breaks.

Complexity

$O(n \text{ alphabet})$ time and memory.

BFS order and correctness details

Queue accepted SAM paths, not arbitrary state visits: different path labels can end in the same state but are different candidate words. Process outgoing letters in sorted order. The first absent transition encountered is shortest because of BFS depth and lexicographically smallest because parents and edges are expanded in lexicographic level order.

Marking states as visited is wrong here—it collapses distinct words sharing an end-position class. Tests should include a missing one-letter word, an alphabet whose every letter occurs, and several shortest absent words with a tie.

Reference C++

```
#include <bits/stdc++.h>
using namespace std;
struct SAM {
    struct Node {
        array<int, 26> next;
        int link = -1, len = 0;
        long long occ = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> st;
    int last = 0;
    SAM() { st.emplace_back(); }
    void add(int c) {
        int cur = st.size();
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].occ = 1;
        int p = last;
        while (p != -1 && st[p].next[c] == -1)
            st[p].next[c] = cur, p = st[p].link;
        if (p == -1)
            st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len)
                st[cur].link = q;
            else {
                int clone = st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;
                st[clone].occ = 0;
                while (p != -1 && st[p].next[c] == q)
                    st[p].next[c] = clone, p = st[p].link;
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
    SAM(const string &s) : SAM() {
        for (char c : s)
            add(c - 'a');
    }
    vector<int> order() const {
        vector<int> o(st.size());
        iota(o.begin(), o.end(), 0);
        sort(o.begin(), o.end(),
            [&](int a, int b) { return st[a].len < st[b].len; });
    }
};
```

```

    return o;
}
void occurrences() {
    auto o = order();
    for (int i = (int)o.size() - 1; i; --i)
        st[st[o[i]].link].occ += st[o[i]].occ;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SAM a(s);
    int n = a.st.size();
    vector<int> par(n, -2), letter(n);
    queue<int> q;
    par[0] = -1;
    q.push(0);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int c = 0; c < 26; ++c) {
            int u = a.st[v].next[c];
            if (u == -1) {
                string ans(1, char('a' + c));
                for (int x = v; x; x = par[x])
                    ans += char('a' + letter[x]);
                reverse(ans.begin(), ans.end());
                cout << ans << '\n';
                return 0;
            }
            if (par[u] == -2) {
                par[u] = v;
                letter[u] = c;
                q.push(u);
            }
        }
    }
}

```

Reference Python

```

class SAM:
    def __init__(self, s):
        self.next = [{}]
        self.link = [-1]
        self.length = [0]
        self.occ = [0]
        last = 0
        for char in s:
            current = len(self.next)
            self.next.append({})
            self.link.append(0)
            self.length.append(self.length[last] + 1)
            self.occ.append(1)
            parent = last
            while parent != -1 and char not in self.next[parent]:
                self.next[parent][char] = current
                parent = self.link[parent]
            if parent == -1:

```

```

        self.link[current] = 0
    else:
        target = self.next[parent][char]
        if self.length[parent] + 1 == self.length[target]:
            self.link[current] = target
        else:
            clone = len(self.next)
            self.next.append(self.next[target].copy())
            self.link.append(self.link[target])
            self.length.append(self.length[parent] + 1)
            self.occ.append(0)
            while (
                parent != -1 and self.next[parent].get(char) == target
            ):
                self.next[parent][char] = clone
                parent = self.link[parent]
            self.link[target] = self.link[current] = clone
    last = current

def order(self):
    return sorted(range(len(self.next)), key=self.length.__getitem__)

def occurrences(self):
    order = self.order()
    for state in reversed(order[1:]):
        self.occ[self.link[state]] += self.occ[state]

def main():
    import collections
    import sys

    automaton = SAM(sys.stdin.buffer.readline().decode().strip())
    parent = [None] * len(automaton.next)
    letter = [""] * len(automaton.next)
    parent[0] = -1
    queue = collections.deque([0])
    while queue:
        state = queue.popleft()
        for code in range(26):
            char = chr(ord("a") + code)
            if char not in automaton.next[state]:
                answer = [char]
                while state:
                    answer.append(letter[state])
                    state = parent[state]
                print("".join(reversed(answer)))
                return
            target = automaton.next[state][char]
            if parent[target] is None:
                parent[target] = state
                letter[target] = char
                queue.append(target)

if __name__ == "__main__":
    main()

```